- `abscissa angabscissa(circle c, point M)`
- `abscissa angabscissa(ellipse el, point M, polarconicroutine polarconicroutine=currentpolarconicroutine)`
- `abscissa angabscissa(hyperbola h, point M, polarconicroutine polarconicroutine=currentpolarconicroutine)`
- `abscissa angabscissa(parabola p, point M)`

"Nodal abscissa"

- `abscissa nodabscissa(line l, point M)`
- `abscissa nodabscissa(ellipse el, point M)`
- `abscissa nodabscissa(parabola p, point M)`

10.3. Operators

`abscissa operator +(real x, explicit abscissa a)`

Return the copy of `a` with abscissa `x+a.x`.

The following operators are also efined:

```
operator +(explicit abscissa,real)
operator -(real,explicit abscissa)
operator -(explicit abscissa,real)
operator -(explicit abscissa a)
operator *(real x, explicit abscissa a)
operator *(explicit abscissa a, real x)
operator /(real x, explicit abscissa a)
operator /(explicit abscissa a, real x).
```

11. Triangles

11.1. Structure

The `triangle` structure is more complex than the previous ones already developed in this document. Indeed it defines new types which create instances of some objects which are inextricably linked to a triangle. Moreover these objects possess the reference of the associated triangle. In other words, an object `TR` of type `triangle` contains as a structure member the object `VA` of type `vertex` which is attained via the code `TR.VA` and the object `TR.VA` contains the object `t` of type `triangle` which value is `TR`; it follows that the value of `TR.VA.t` is `TR`.

Since an object of type `vertex` is a triangle vertex, it is a straightforward task to write a routine that returns for instance the internal bisector of the angle of a triangle: it is sufficient to give an object of type `vertex` since this object contains the reference of the triangle of which it is a vertex.

Here is a simplified version of the `triangle` structure; the complete structure is detailed separately.

```
struct triangle {
  restricted point A, B, C;

  struct vertex {
    int n;
    triangle t; }

  restricted vertex VA, VB, VC;

  struct side {
    int n;
    triangle t; }

  side AB, BC, CA, BA, AC, CB; }
```

`A`, `B` and `C` represent the points marking the triangle vertices;

struct vertex defines the **vertex** structure which allows to create the instance of an object which represents the vertex of a triangle. Since we need to manipulate this structure in a very few cases, it is useful to know his properties.

The property **n** allows to associated a vertex to a point, of type **point**, marking this vertex:
if **n** = 1, the vertex is associated to the point **A**;
if **n** = 2, the vertex is associated to the point **B**;
if **n** = 3, the vertex is associated to the point **C**;
if **n** = 4, the vertex is associated to the point **A**;
etc. The value of the property **t** is "the object of type **triangle** to which the vertex belongs".
For more details on the use of this structure see Triangles vertices.

VA, VB et VC represent the vertices of the triangle in an abstract way as opposed to the objects **point A**, **B** and **C** that are the points which mark each vertex; see Section Triangles vertices.

struct side defines the **side** structure which allows to create the instance of an object which represents the side of a triangle. Since we need to manipulate this structure in a very few cases, it is useful to know his properties.

The property **n** allows to associated the object of type **side** to a triangle's side;
if **n** = 1, the side represents **AB** oriented from **A** to **B**;
if **n** = 2, the side represents **BC** oriented from **B** to **C**;
if **n** = 3, the side represents **CA** oriented from **C** to **A**;
if **n** = 4, the side represents **AB**;
etc.

if **n** is negative the orientation is reversed.
The value of the property **t** is "the object of type **triangle** to which the side belongs".
Routines on **side** are described into Section Triangles sides.

AB, BC, CA, BA, AC et CB represent the sides of the triangle in a abstract way as opposed to the objects **line line(TR.AB)**, **line(TR.BC)**, **line(TR.CA)**, etc, that are the lines marking the sides of the triangle **TR**; see Section Triangles sides.

11.2. Defining and drawing a triangle

This section is devoted to describe the basic routines to define and to draw a triangle. Another routines will be given as of reading.

- ```
  void label(picture pic=currentpicture, Label LA="$A$",
  Label LB="$B$", Label LC="$C$",
  triangle t,
  real alignAngle=0,
  real alignFactor=1,
  pen p=nullpen, filltype filltype=NoFill)
  ```

  Draw the labels **LA**, **LB** and **LC** at the vertices of the triangle **t**. The position depends on the internal bisector of the corresponding vertex. The parameters **alignAngle** and **alignFactor** allow to modify the direction and the length of the alignment.

- ```
  void show(picture pic=currentpicture,
  Label LA="$A$", Label LB="$B$", Label LC="$C$",
  Label La="$a$", Label Lb="$b$", Label Lc="$c$",
  triangle t, pen p=currentpen, filltype filltype=NoFill)
  ```

  Draw the triangle **t** and place the labels at the vertices of the triangle and the lengths of the sides. When coding this routine is useful to locate the vertices **t.A**, **t.B** and **t.C**.

- ```
  void draw(picture pic=currentpicture, triangle t,
  pen p=currentpen, marker marker=nomarker)
  ```

  Draw the triangle **t**; sides are drawn as segments.

- ```
  void drawline(picture pic=currentpicture, triangle t, pen p=currentpen)
  ```

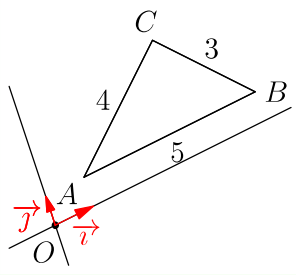
  Draw the triangle **t**; sides are drawn as lines.

- ```
  triangle triangle(point A, point B, point C)
  ```

  Return the triangle whose vertices are **A**, **B** et **C**.
  ```

- `triangle triangleabc(real a, real b, real c, real angle=0, point A=(0,0))`

  Return the triangle `ABC` such that `BC` = `a`, `AC` = `b`, `AB` = `c` and $\left(\overrightarrow{i}\,;\overrightarrow{AB}\right)$ = `angle`.



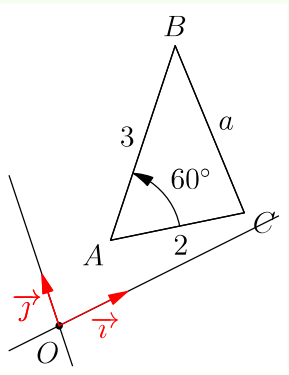

```
import geometry;
size(4cm);

currentcoordsys=cartesiansystem(i=(1,0.5), j=(-0.25,.75));
show(currentcoordsys);

triangle t=triangleabc(3,4,5, (1,1));
show(La="3", Lb="4", Lc="5", t);
```

- `triangle triangleAbc(real alpha, real b, real c, real angle=0, point A=(0,0))`

  Return the triangle `ABC` such that $\left(\overrightarrow{AB}\,;\overrightarrow{AC}\right)$ = `alpha`, `AC` = `b`, `AB` = `c` et $\left(\overrightarrow{i}\,;\overrightarrow{AC}\right)$ = `angle`.




```
import geometry;

size(5cm);

currentcoordsys=cartesiansystem(i=(1,0.5),j=(-0.25,.75));
show(currentcoordsys);

triangle t=triangleAbc(-60,2,3,angle=45,(1,1));
show(Lb="2", Lc="3",t);
markangle("$60^\circ$",t.C,t.A,t.B, Arrow);
```

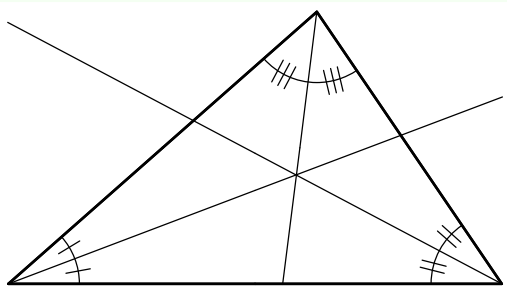
- `triangle triangle(line l1, line l2, line l3)`

  Return the triangle whose sides are `l1`, `l2` and `l3`.

11.3. Vertices triangles

Let `t` be an object of type `triangle`. Its properties `t.VA`, `t.VB` and `t.VC` are of type `vertex` and describe the vertices of the triangle `t`. The package *geometry.asy* allows routines with a vertex of triangle as parameter without specifying explicitly the triangle to which it refers.

As an example in the following code the routine `line bisector(vertex V, real angle=0)` returns the image by the rotation of center `V` and angle `angle` of the internal bisector passing through `V`.



```
size(7cm); import geometry;
triangle t=triangleabc(4,5,6);
drawline(t, linewidth(bp));
line ba=bisector(t.VA), bb=bisector(t.VB);
line bc=bisector(t.VC); draw(ba^^bb^^bc);
markangle((line) t.AB, (line) t.AC, StickIntervalMarker(2,1));
markangle((line) t.BC, (line) t.BA, StickIntervalMarker(2,2));
markangle((line) t.CA, (line) t.CB, StickIntervalMarker(2,3));
```

Here we have some routines and basic operators on the objects of type `vertex`

- `point operator cast(vertex V)`

  Allow the casting of `vertex` in `point`.

- `point point(explicit vertex V)`

  Return the object of type `point` corresponding to the object `V` of type `vertex`. The code `point(V)` is equivalent to the code `(point)V` which forces the casting of `vertex` in `point`.

- `vector dir(vertex V)`

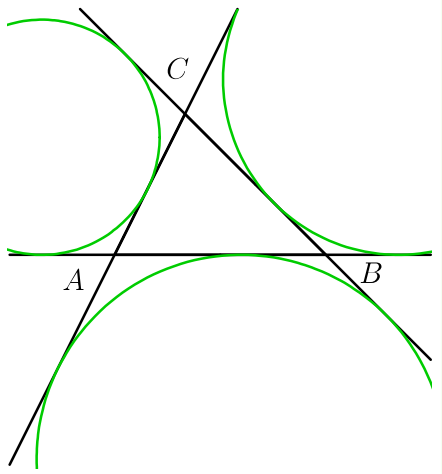
  Return the unit vector on the internal bisector of the angle in `V` and oriented outwards the triangle to which `V` refers. This routine is especially useful to place the labels at the vertices of a triangle.

Another routines with object of type `vertex` as parameter are described in the following section and together with routines regarding triangles.

11.4. Sides triangles

Let `t` be an object of type `triangle`. Its properties `t.AB`, `t.BC`, `t.CA`, `t.BA`, `t.AC` and `t.CB`, of type `side` represents the sides of the triangle `t`. The package *geometry.asy* allows routines with a side of triangle as parameter without specifying explicitly the triangle to which it refers.

As an example in the following code the routine `circle excircle(side s)` returns the excircle of the triangle to which `s` refers and tangent to `s`.




```
import geometry;
size(6cm,0);

triangle t=triangle((-1,0), (2,0), (0,2));

drawline(t, linewidth(bp));
label(t,alignFactor=4);

clipdraw(excircle(t.AB), bp+0.8green);
clipdraw(excircle(t.BC), bp+0.8green);
clipdraw(excircle(t.AC), bp+0.8green);

draw(box((-2.5,-3), (3.5,3.5)), invisible);
```

Here we have some routines and basic operators on the objects of type `side`:

- `line operator cast(side side)`

  Allow the casting of `side` in `line`.

- `line line(explicit side side)`

  Return the object of type `line` corresponding to the object `side` of type `side`. The code `line(S)` is equivalent to `(line)S` which forces the casting of `side` to `line`.

- `segment segment(explicit side side)`

  Return the object of type `segment` corresponding to the object `side` of type `side`. The code `segment(S)` is equivalent to `(segment)S` which forces the casting of `side` in `segment`.

- `side opposite(vertex V)`

  Return the opposite side to `V` about the triangle to which `V` refers.

- `vertex opposite(side side)`

  Return the opposite vertex to `side` into the triangle to which `side` refers.

The another routines with object of type `side` as parameter are described together with routines regarding triangles

11.5. Operators

The single operator which can be applied to `triangle` is

`triangle operator *(transform T, triangle t)`

that allows the code `transform*triangle`.

11.6. Another routines

- `point orthocentercenter(triangle t)`

  Return the orthocenter of the triangle `t`.

- `point foot(vertex V)`

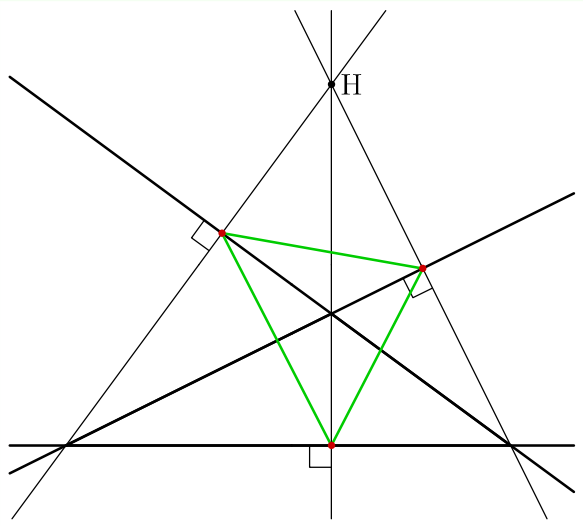
  Return the foot of the altitude from `V`. The routine `point foot(side side)` is also available.

- `line altitude(vertex V)`

  Return the altitude from `V`. The routine `line altitude(side side)` is also available.

- `triangle orthic(triangle t)`

  Return the orthic triangle of `t`; its vertices are the feet of the altitudes of `t`.




```
size(8cm);
import geometry;

triangle t=triangleabc(3,4,6);
drawline(t, linewidth(bp));
line hc=altitude(t.AB), hb=altitude(t.AC);
line ha=altitude(t.BC); draw(hc^^hb^^ha);
dot("H", orthocentercenter(t));

perpendicularmark(t.AB,hc,quarter=-1);
perpendicularmark(t.AC,hb,quarter=-1);
perpendicularmark(t.BC,ha);

triangle ort=orthic(t);
draw(ort,bp+0.8*green); dot(ort, 0.8*red);
addMargins(1cm,1cm);
```

- `point midpoint(side side)`

  Return the midpoint of `side`.

- `point centroid(triangle t)`

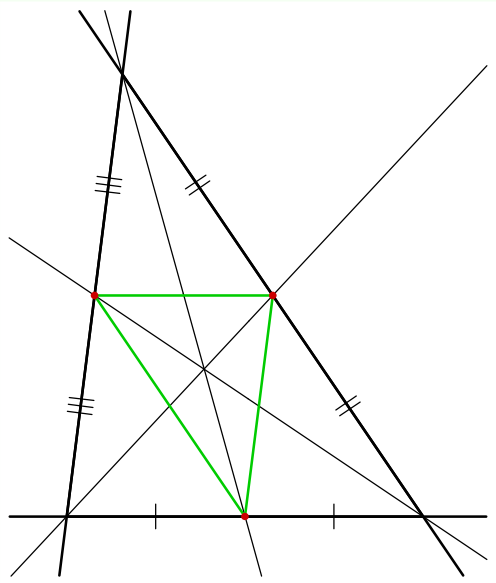
  Return the centroid of the triangle `t`.

- `line median(vertex V)`

  Return the median from `V`. The routine `line median(side side)` is also available.

- `triangle medial(triangle t)`

  Return the medial triangle of `t` (its vertices are the midpoints of `t`).



```
size(8cm);
import geometry;

triangle t=triangleabc(6,5,4);
drawline(t, linewidth(bp));
line ma=median(t.VA), mb=median(t.VB);
line mc=median(t.VC); draw(ma^^mb^^mc);

draw(segment(t.AB), StickIntervalMarker(2,1));
draw(segment(t.BC), StickIntervalMarker(2,2));
draw(segment(t.CA), StickIntervalMarker(2,3));

triangle med=medial(t);
draw(med,bp+0.8*green); dot(med, 0.8*red);
addMargins(1cm,1cm);
```

- `triangle anticomplementary(triangle t)`

  Return the anticomplementary triangle of `t` (`t` is the median triangle of `anticomplementary(t)`).

- `line bisector(vertex V, real angle=0)`

  Return the image of the internal bisector through `V` under the rotation of center `V` and angle `angle`. An example has already been given.

- `point bisectorpoint(side side)`

  Return the intersection point of `side` with the internal bisector of the opposite angle to `side`.

- `line bisector(side side)`

  Return the perpendicular bisector of the segment corresponding to `side`.

- `point circumcenter(triangle t)`

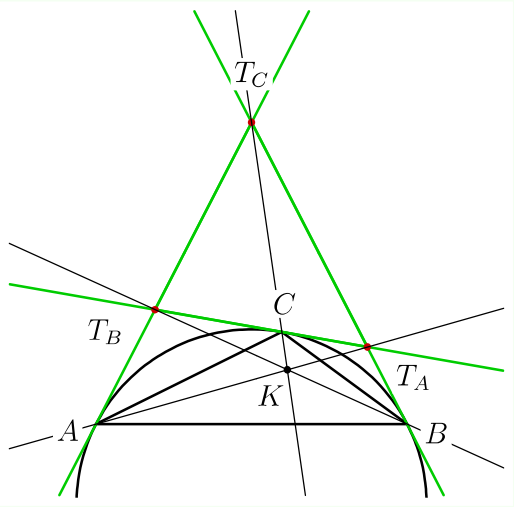
  Return the circumcenter of the triangle `t`.

- `circle circle(triangle t)`

  Return the circumcircle of the triangle `t`. The routine `circumcircle(triangle t)` is an alias.

- `triangle tangential(triangle t)`

  Return the tangential triangle of `t`, that is the triangle whose sides are tangent to the circumcircle of `t`.




```
size(7cm); import geometry;
triangle t=triangleabc(3,4,6);
draw(t, linewidth(bp));
clipdraw(circle(t), linewidth(bp));
triangle itr=tangential(t);
drawline(itr, bp+0.8*green); dot(itr, 0.8*red);
line syma=line(itr.A,t.A), symb=line(itr.B,t.B);
line symc=line(itr.C,t.C); draw(syma^^symb^^symc);
dot("$K$", intersectionpoint(syma,symb),
    2*dir(-120));
label(t,alignFactor=2,UnFill);
label("$T_A$","$T_B$","$T_C$", itr, alignFactor=4,
      UnFill);
addMargins(1cm,1cm);
```

- `point incenter(triangle t)`

  Return the incenter of `t`, that is the center of the incircle of `t`.

- `real inradius(triangle t)`

  Return the radius of the incircle of `t` (also known as the inradius).

- `circle incircle(triangle t)`

  Return the incircle of `t`.

- `triangle intouch(triangle t)`

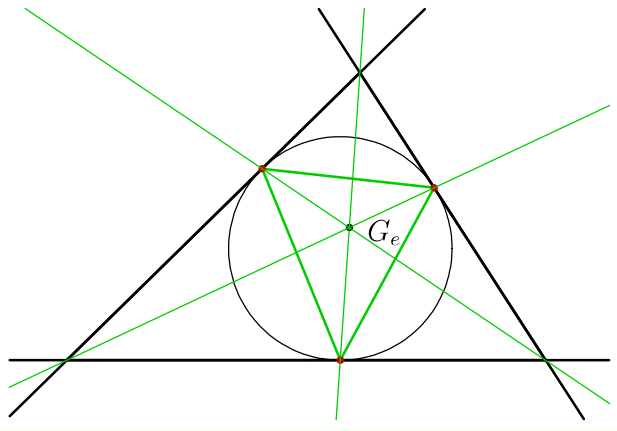
  Return the contact triangle or intouch triangle that is the triangle whose vertices are the three points where the incircle touches the triangle `t`.

- `point intouch(side side)`

  Return the contact point of the side `side` with the incircle to which `side` refers.

- `point gergonne(triangle t)`

  Return the GERGONNE point of the triangle `t`.




```
size(8.5cm,0);import geometry;
triangle t=triangleabc(5,6,7);
drawline(t, linewidth(bp));
draw(incircle(t));
triangle itr=intouch(t);
draw(itr,bp+0.8*green); dot(itr, 0.8*red);
point Ge=gergonne(t);
dot("$G_e$", Ge, 2*dir(-10));
draw(line(Ge,t.A), 0.8*green);
draw(line(Ge,t.B), 0.8*green);
draw(line(Ge,t.C), 0.8*green);
addMargins(1cm,1cm);
```

- `point excenter(side side)`

  Return the center of the excircle of the triangle to which `side` refers and tangent to `side`.

- `real exradius(side side)`

  Return the radius of the excircle of the triangle to which `side` refers and tangent to `side`.

- `circle excircle(side side)`

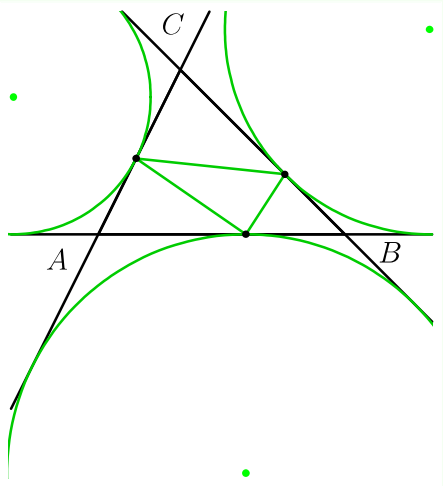
  Return the excircle of the triangle to which `side` refers and tangent to `side`.

- `triangle extouch(triangle t)`

  Return the extouch triangle of `t` that is the triangle whose vertices are the contact points of the three excircles to `t` with its sides.

- `point extouch(side side)`

  Return the contact point of the side `side` with the excircle given by `excircle(side)`.




```
import geometry; size(6cm,0);

triangle t=triangle((-1,0), (2,0), (0,2));
drawline(t, linewidth(bp));
label(t,alignFactor=4);

circle c1=excircle(t.AB), c2=excircle(t.BC);
circle c3=excircle(t.AC);
clipdraw(c1, bp+0.8green);
clipdraw(c2, bp+0.8green);
clipdraw(c3, bp+0.8green);
dot(c1.C^^c2.C^^c3.C, green);
draw(extouch(t), bp+0.8green, dot);
```

- `point symmedian(triangle t)`

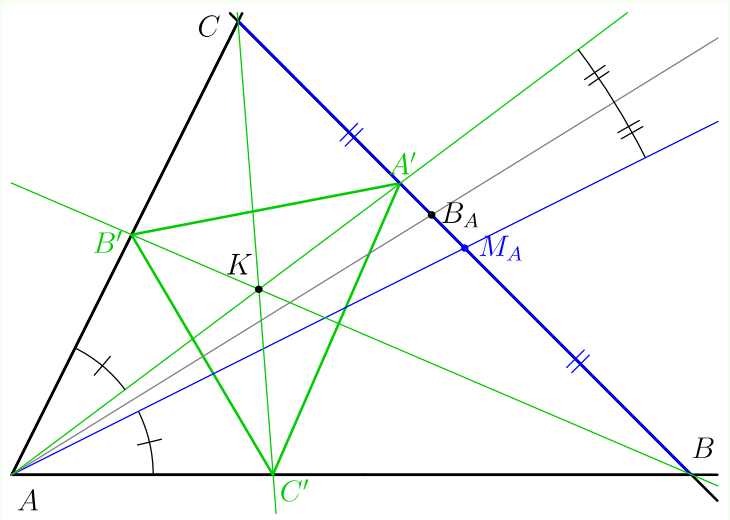
  Return the symmedian point (also called the LEMOINE point) of the triangle `t`.

- `point symmedian(side side)`

  Return the symmedian point of the side `side`.

- `line symmedian(vertex V)`

  Return the symmedian line of the triangle to which `V` refers and passing through `V`.




```
import geometry; size(10cm,0);
triangle t=triangle((-1,0), (2,0), (0,2)); drawline(t, linewidth(bp));
label(t,alignFactor=2, alignAngle=90);
triangle st=symmedial(t); draw(st, bp+0.8green);
label("$A'$", "$B'$", "$C'$", st, alignAngle=45, 0.8green);
line mA=median(t.VA); draw(mA, blue); dot("$M_A$",midpoint(t.BC), 1.5E, blue);
draw(segment(t.BC), bp+blue, StickIntervalMarker(2,2,blue));
line bA=bisector(t.VA); draw(bA, grey); dot("$B_A$", bisectorpoint(t.BC));
line sA=symmedian(t.VA); draw(sA, 0.8*green);
draw(symmedian(t.VB), 0.8*green); draw(symmedian(t.VC), 0.8*green);
point sP=symmedian(t); dot("$K$", sP, 2*dir(125));
markangle(sA, (line) t.AC, radius=2cm, StickIntervalMarker(1,1));
markangle((line) t.AB, mA, radius=2cm, StickIntervalMarker(1,1));
markangle(mA, sA, radius=10cm, StickIntervalMarker(2,2));
```

- `point cevian(side side, point P)`

  Return the Cevian point of `P` belonging to the side `side`.

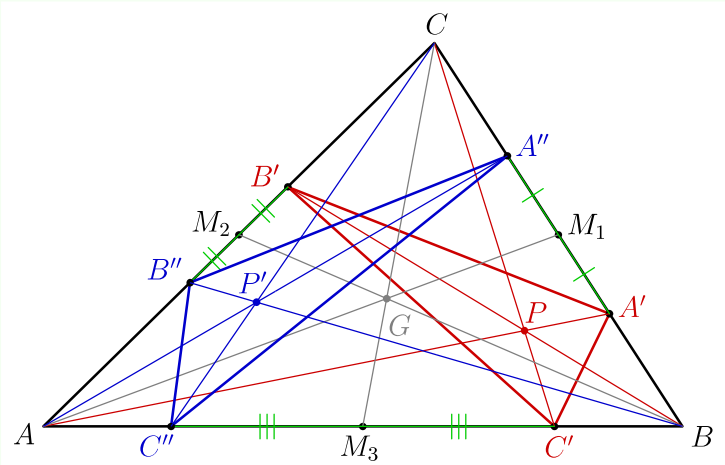
- `triangle cevian(triangle t, point P)`

  Return the Cevian triangle of `P`.

- `line cevian(vertex V, point P)`

  Return the Cevian line of `P` passing through `V` in the triangle to which `V` refers.

  The following example is an illustration of the property "*if the triangle $A'B'C'$ is a* Cevian *triangle of the triangle ABC then the triangle $A''B''C''$ whose vertices are the symmetric of A', B' et C' about the respective midpoints is also a* Cevian *triangle*".




```
import geometry; size(10cm,0);
triangle t=triangleabc(5,6,7); label(t); draw(t, linewidth(bp));
point P=0.6*t.B+0.25*t.C; dot("$P$", P, dir(60), 0.8*red);
triangle C1=cevian(t, P);
label("$A'$","$B'$","$C'$", C1, 0.8*red); draw(C1, bp+0.8*red, dot);
draw(t.A--C1.A, 0.8*red); draw(t.B--C1.B, 0.8*red); draw(t.C--C1.C, 0.8*red);

point Ma=midpoint(t.BC), Mb=midpoint(t.AC), Mc=midpoint(t.BA);
dot("$M_1$",Ma,-dir(t.VA)); dot("$M_2$",Mb,-dir(t.VB)); dot("$M_3$",Mc,-dir(t.VC));
draw(t.A--Ma^^t.B--Mb^^t.C--Mc, grey); dot("$G$", centroid(t), 2*dir(-65), grey);

point App=rotate(180,Ma)*C1.A, Bpp=rotate(180,Mb)*C1.B, Cpp=rotate(180,Mc)*C1.C;
draw(C1.A--App, 0.8*green, StickIntervalMarker(2,1,0.8*green));
draw(C1.B--Bpp, 0.8*green, StickIntervalMarker(2,2,0.8*green));
draw(C1.C--Cpp, 0.8*green, StickIntervalMarker(2,3,0.8*green));

triangle C2=triangle(App,Bpp,Cpp);
label("$A''$","$B''$","$C''$", C2, 0.8*blue); draw(C2, bp+0.8*blue, dot);
segment sA=segment(t.A,C2.A), sB=segment(t.B,C2.B);
point PP=intersectionpoint(sA,sB);
dot("$P'$", PP, dir(100), 0.8*blue);
draw(sA, 0.8*blue); draw(sB, 0.8*blue); draw(segment(t.C,C2.C), 0.8*blue);
```

- `line isotomic(vertex V, point M)`

  Return the isotomic line through `V` and relative to `M` about the triangle which `V` refers.

- `point isotomicconjugate(triangle t, point M)`

  Return the isotomic conjugate of `M` relatively to `t`.

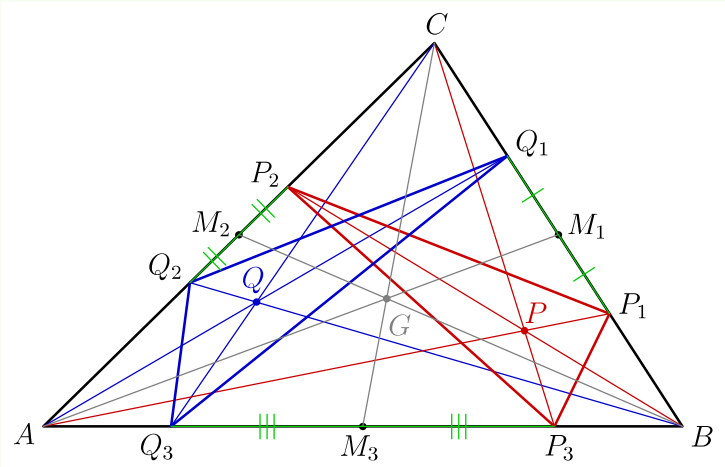
- `point isotomic(side side, point M)`

  Return the intersection point of the isotomic line of `M`, about the triangle which `side` refers, with the side `side`.

- `triangle isotomic(triangle t, point M)`

  Return the triangle whose the vertices are the intersection points of the isotomic lines relative to `M` about `t` with the sides of `t`. So, in the previous picture, the triangle $A''B''C''$ is the isotomic triangle relative to `P`.

  Below, the same figure obtained using routines `isotomic` gains in brevity.




```
import geometry; size(10cm,0);
triangle t=triangleabc(5,6,7); label(t); draw(t, linewidth(bp));
point P=0.6*t.B+0.25*t.C; dot("$P$", P, dir(60), 0.8*red);
draw(segment(isotomic(t.VA,P))^^segment(isotomic(t.VB,P))^^segment(isotomic(t.VC,P)),
     0.8*blue);
draw(segment(cevian(t.VA,P))^^segment(cevian(t.VB,P))^^segment(cevian(t.VC,P)),
     0.8*red);
triangle t1=cevian(t,P); label("$P_1$", "$P_2$", "$P_3$", t1); draw(t1, bp+0.8*red);
triangle t2=isotomic(t,P); label("$Q_1$", "$Q_2$", "$Q_3$", t2); draw(t2, bp+0.8*blue);
dot("$Q$", isotomicconjugate(t,P), dir(100), 0.8*blue);

point Ma=midpoint(t.BC), Mb=midpoint(t.AC), Mc=midpoint(t.BA);
dot("$M_1$",Ma,-dir(t.VA)); dot("$M_2$",Mb,-dir(t.VB)); dot("$M_3$",Mc,-dir(t.VC));
draw(t.A--Ma^^t.B--Mb^^t.C--Mc, grey); dot("$G$", centroid(t), 2*dir(-65), grey);
draw(t1.A--t2.A, 0.8*green, StickIntervalMarker(2,1,0.8*green));
draw(t1.B--t2.B, 0.8*green, StickIntervalMarker(2,2,0.8*green));
draw(t1.C--t2.C, 0.8*green, StickIntervalMarker(2,3,0.8*green));
```

- `point isogonalconjugate(triangle t, point M)`

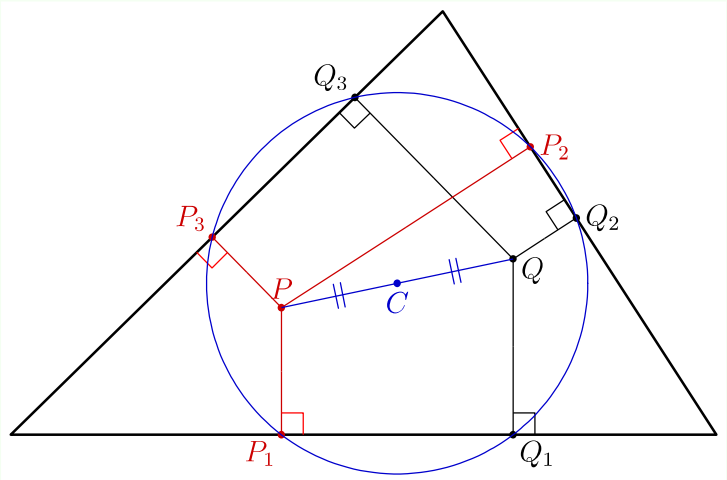
  Return the isogonal conjugate of `M` about `t`.

- `point isogonal(side side, point M)`

  Return the intersection point of the isogonal line of `M`, according to the triangle which `side` refers, with the side `side`.

- `triangle isogonal(triangle t, point M)`

  Return the triangle whose the vertices are the intersection points with the sides of `t` of the isogonal lines relative to `M` about `t`.
  The following example illustrates the property *"the pedal triangles of two isogonal points P and Q are inscribed in the same circle centered at the midpoint of the segment line $[PQ]$."*




```
import geometry; size(10cm,0);
triangle t=triangleabc(5,6,7); draw(t, linewidth(bp));
point P=0.5*t.B+0.3*(t.C-t.B); dot("$P$", P, N, 0.8*red);

point Q=isogonalconjugate(t,P); dot("$Q$", Q, dir(-30));
point Q1=projection(t.AB)*Q; segment sq1=segment(Q,Q1);
point Q2=projection(t.BC)*Q; segment sq2=segment(Q,Q2);
point Q3=projection(t.AC)*Q; segment sq3=segment(Q,Q3);
draw(sq1); draw(sq2); draw(sq3);
dot("$Q_1$", Q1, SE); dot("$Q_2$", Q2); dot("$Q_3$", Q3, NW);

point P1=projection(t.AB)*P; segment sp1=segment(P,P1);
point P2=projection(t.BC)*P; segment sp2=segment(P,P2);
point P3=projection(t.AC)*P; segment sp3=segment(P,P3);
draw(sp1, 0.8*red); draw(sp2, 0.8*red); draw(sp3, 0.8*red);
dot("$P_1$",P1,SW,0.8*red); dot("$P_2$",P2,0.8*red); dot("$P_3$",P3,NW,0.8*red);

perpendicularmark(t.AB,sq1); perpendicularmark(t.BC,sq2);
perpendicularmark(reverse(t.AC),sq3); perpendicularmark(t.AB,sp1, red);
perpendicularmark(t.BC,sp2, red); perpendicularmark(reverse(t.AC),sp3, red);

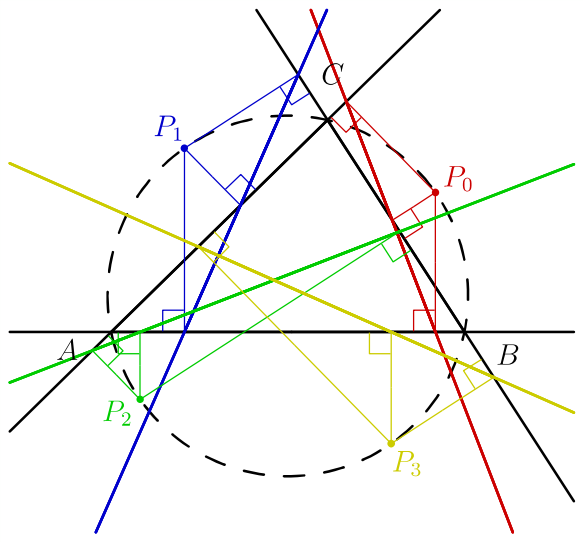
circle C=circle(Q1,Q2,Q3); draw(C, 0.8*blue);
draw(segment(Q,P), 0.8*blue, StickIntervalMarker(2,2, 0.8*blue));
dot("$C$", C.C, S, 0.8*blue);
```

- `triangle pedal(triangle t, point M)`

  Return the pedal triangle of `M` about `t`.

- `line pedal(side side, point M)`

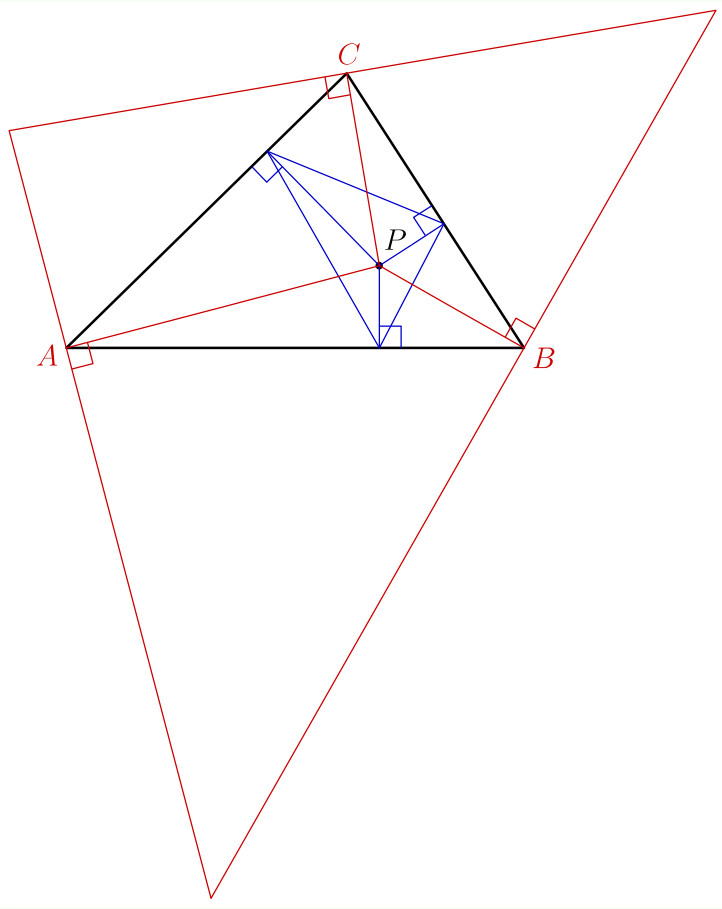
  Return the line through `M` and through the foot of the perpendicular from `M` to the side line `side`.
  The following example shows a few lines of SIMSON; note the use of methods `t.side(int)` and `t.vertex(int)` that allows to obtain by theirs numbers the sides and the vertices of the triangle `t`.




```
import geometry; size(8cm,0);
triangle t=triangleabc(5,6,7);
label(t, alignFactor=4);
drawline(t, linewidth(bp));
circle C=circle(t); draw(C, bp+dashed);
pen[] p=new pen[] {0.8*red,0.8*blue,
                   0.8*green, 0.8*yellow};
for (int i=0; i < 4; ++i) {
  real x=35+i*90; point P=angpoint(C,x);
  dot("$P_"+(string)i+"$",P,dir(x),p[i]);
  for (int j=1; j < 4; ++j) {
    segment Sg=segment(pedal(t.side(j),P));
    draw(Sg,p[i]);
    markrightangle(P,Sg.B,t.vertex(j),p[i]);
  }
  drawline(pedal(t,P), bp+p[i]);
}
addMargins(1cm,1cm);
```

- `triangle antipedal(triangle t, point M)`

  Return the triangle whose the pedal triangle of `M` is `t`.




```
import geometry; size(10cm,0);
triangle t=triangleabc(5,6,7);
label(t); draw(t, linewidth(bp));
point P=0.5*t.B+0.3*t.C;
dot("$P$", P, 2*dir(60));

triangle Pt=pedal(t,P);
currentpen=0.8*blue; draw(Pt);
segment psA=segment(P,Pt.A);
segment psB=segment(P,Pt.B);
segment psC=segment(P,Pt.C);
draw(psA); draw(psB); draw(psC);
perpendicularmark(t.BC,psA);
perpendicularmark(t.CA,psB);
perpendicularmark(t.AB,psC);

triangle APt=antipedal(t, P);
currentpen=0.8*red; draw(APt);
segment apsA=segment(P,t.A);
segment apsB=segment(P,t.B);
segment apsC=segment(P,t.C);
draw(apsA); draw(apsB); draw(apsC);
perpendicularmark(APt.BC,apsA);
perpendicularmark(APt.CA,apsB);
perpendicularmark(APt.AB,apsC);
```

11.7. Trilinear coordinates

The type `trilinear`, whose the structure is given further, allows to instantiate an object representing the trilinear coordinates `a:b:c` with respect to the triangle `t`.

```
struct trilinear
{
  real a,b,c;
  triangle t;
}
```

To define the trilinear coordinates `a:b:c` with respect to a triangle `t` one can use the routine

```
trilinear trilinear(triangle t, real a, real b, real c)
```

It is also possible to obtain the trilinear coordinates of a point through the routine

```
trilinear trilinear(triangle t, point M)
```

It is also possible to define trilinear coordinated through a triangle center function f and three parameters `a`, `b` and `c` using the following routine:

```
trilinear trilinear(triangle t, centerfunction f,
real a=t.a(), real b=t.b(), real c=t.c())
```

where the type `centerfunction` represents a real function with three real variables.

The casting of an object of type `trilinear` to type `point` may be done, as usual, through two ways: through the routine `point(trilinear)` or through syntax casting `(point) trilinear`.

For example, using the trilinear coordinates of the isotomic conjugate of a point, here's how it is defined the routine `isotomicconjugate`:

```
point isotomicconjugate(triangle t, point M)
{
  trilinear tr=trilinear(t,M);
  return point(trilinear(t,1/(t.a()^2*tr.a),1/(t.b()^2*tr.b),1/(t.c()^2*tr.c)));
}
```

12. Inversions

The type `inversion`, whose the structure is given below, allows to instantiate an inversion with center `C` and radius `k`

```
struct inversion
{
  point C;
  real k;
}
```

12.1. Define an inversion

The following routines and operators allow to define an inversion.

- `inversion inversion(real k, point C)`

  Return the inversion with center `C` and radius `k`. The routine `inversion(point C, real k)` is also available.

- `inversion inversion(circle c1, circle c2, real sgn=1)`

  – if `sgn` is non-zero, this routine returns the inversion whose the radius is the same sign as `sgn` and transforming `c1` to `c2`;

  – if `sgn` is zero, this routine returns the inversion centered at the foot of the radical axis and leaving globally invariants each of the two circles `c1` and `c2`.

  An example using this routine has already been given.

- `inversion inversion(circle c1, circle c2, circle c3)`

  Return the inversion leaving globally invariants the three circles `c1`, `c2` et `c3`.

- `circle operator cast(inversion i)`

  Allow the casting of an object of type `inversion` to `circle`. The returned circle is the principal circle of `i`.
  It may also force the casting through the routine `circle circle(inversion i)`.

- `inversion operator cast(circle c)`

  Allow the casting of a `circle` to an `inversion`. The returned inversion leaves globally invariant `c`, with center the center of `c` and the sign of the radius is the same as the radius of `c`.
  It may also force the casting through the routine `inversion inversion(circle c)`.